

The slide features abstract green geometric shapes. On the left, a thin, light green triangle points downwards. On the right, a complex arrangement of overlapping triangles in various shades of green (from light lime to dark forest green) forms a larger, irregular shape. A thin, light gray line extends from the bottom left towards the right, passing behind the green shapes.

Instruction Set Architecture

Tingting HU

05/05/2026

■ How Instructions Are Encoded

Each instruction is a fixed pattern of bits divided into fields.



← here 16 bits total →

Trade-off in instruction width

Opcode field: identifies the operation (4 bits → up to 16 instructions)

Operand fields: specify registers, memory addresses, or immediate values

More opcode bits → more instructions possible

More operand bits → larger address space or more registers

Fixed-length vs variable-length

Fixed-length: all instructions same size; simpler to decode, may waste bits

Example: RISC-V uses 32-bit fixed

Variable-length: different sizes; more flexible, harder to decode

Example: x86 instructions are 1-15 bytes

Toy 16-bit example - real instruction formats are more complex and vary by ISA.

■ Three Sources of Operands

Every operand the CPU acts on must come from one of these three places.



Register

Small, ultra-fast storage inside the CPU.

~1 cycle · 16-32 per core

```
ADD R3, R1, R2
```

Like a local variable the compiler keeps close at hand.



Memory

Large but slower.

~100 cycles (if not cached!)

```
LOAD R1, [addr]
```

Like accessing a variable stored in RAM.



Immediate

A constant value embedded directly in the instruction.

no fetch - value is in the instruction

```
ADD R1, R1, #10
```

Like writing $x = x + 10$ - the 10 is a constant.

The programmer writes variables and constants; the compiler decides whether they become register operands, memory operands, or immediate values.

Instruction format

Example-1: Given the following instruction format, operands can come from register, main memory or immediate value encode directly in the instruction

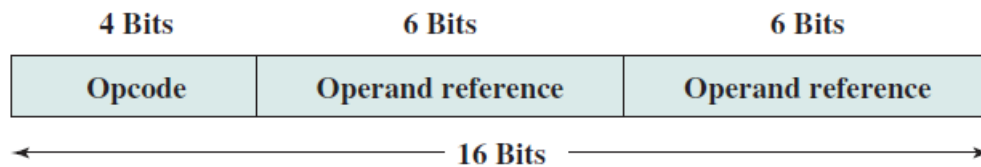


Figure 12.2 A Simple Instruction Format

Q1: if operands are retrieved from main memory, how many different operand addresses can we refer to?

Instruction format

Example-1: Given the following instruction format, operands can come from register, main memory or immediate value encode directly in the instruction

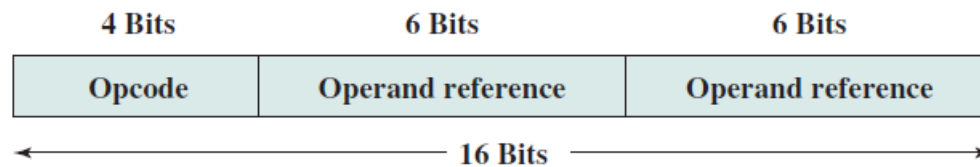


Figure 12.2 A Simple Instruction Format

Q1: if operands are retrieved from main memory, how many different operand addresses can we refer to?

Operand reference is the memory address to retrieve an operand, this is a 2-address instruction → instruction with 2 operands

Each operand field has 6 bits → $2^6 = 64$ different memory addresses, each can refer to memory address 000000 ... 111111) → 64 different operand addresses

Instruction format

Example-1: Given the following instruction format, operands can come from register, main memory or immediate value encode directly in the instruction

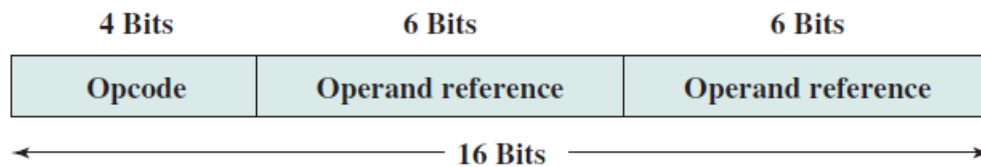


Figure 12.2 A Simple Instruction Format

Q2: if the 2nd operand is immediate value encoded directly in the instruction (assuming unsigned representation), would it be possible to encode immediate value 128₁₀ in the instruction, e.g. `add R1 #128`?

Instruction format

Example-1: Given the following instruction format, operands can come from register, main memory or immediate value encode directly in the instruction

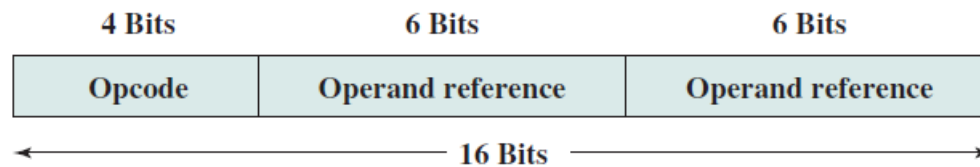


Figure 12.2 A Simple Instruction Format

Q2: if the 2nd operand is immediate value encoded directly in the instruction (assuming unsigned representation), would it be possible to encode immediate value 128_{10} in the instruction, e.g. add R1 #128?

Since operand field has 6 bits $\rightarrow$ the maximum value it can encode is $111111_2 = 2^6 - 1 = 63_{10}$. Hence, it is not possible to encode an immediate value 128_{10} directly in the instruction.

Instruction format

Example-1: Given the following instruction format, operands can come from register, main memory or immediate value encode directly in the instruction

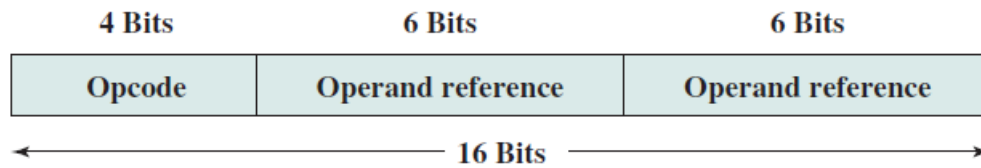


Figure 12.2 A Simple Instruction Format

Q3: which instruction will be the fastest:

- 1) add R1, #12
- 2) add R1, R2
- 3) add R1, X (X is a variable stored in main memory)

Instruction format

Example-1: Given the following instruction format, operands can come from register, main memory or immediate value encode directly in the instruction

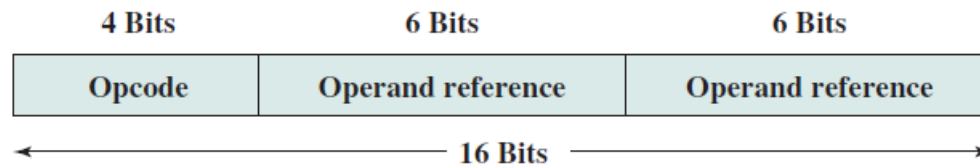


Figure 12.2 A Simple Instruction Format

Q3: which instruction will be the fastest:

- 1) add R1, #12
- 2) add R1, R2
- 3) add R1, X (X is a variable stored in main memory)

Immediate operand (add R1, #12)

- ✓ Fastest in general because the value is already inside the instruction.
- ✓ No extra data fetch needed beyond the instruction itself.

Register operand (add R1, R2)

- ✓ Requires reading from a register (very fast, but still an extra step compared to immediate).

Memory operand (add R1, X)

- ✓ Slowest because it needs a memory access, which is much slower than register access.

Instruction format

Example-2: Given the following instruction format,

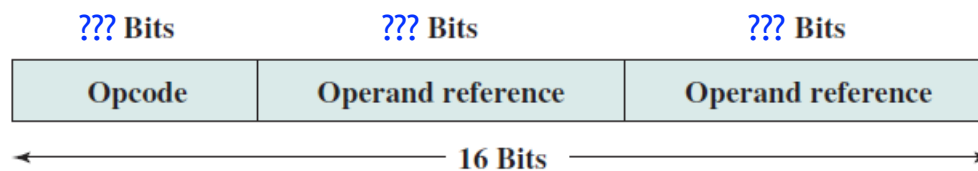


Figure 12.2 A Simple Instruction Format

Q1: if we would like to support 64 different instructions, how many bits need to be reserved for the opcode field? Explain why?

Q2: build on Q1 and in addition, if the two operand fields have the same number of bits, how many different operand addresses can we refer to?

Instruction format

Example-2: Given the following instruction format:

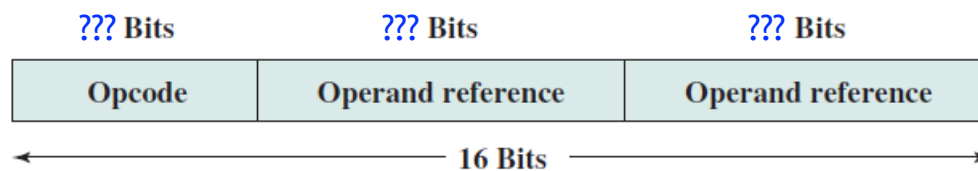


Figure 12.2 A Simple Instruction Format

Q1: if we would like to support 64 different instructions, how many bits need to be reserved for the opcode field? Explain why?

64 different instructions → 6 bits needed, $2^6 = 64$

Instruction format

Example-2: Given the following instruction format:

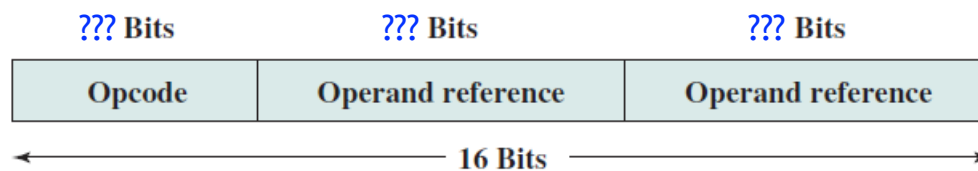


Figure 12.2 A Simple Instruction Format

Q2: build on Q1 and in addition, if the two operand fields have the same number of bits, how many different operand addresses we can refer to?

6 bits for opcode $\rightarrow 16 - 6 = 10$ bits for operands $\rightarrow 5$ bits for each operand
 $\rightarrow$ We can refer to $2^5 = 32$ different operand addresses

■ How Many Operands?

Four classical instruction styles - from most explicit to most compact.

Style	Format	Example: $x = a + b$	Notes
Three-address	OP dest, src1, src2	ADD x, a, b	One instruction does the job
Two-address	OP dest/src1, src2	LOAD R1,a; ADD R1,b	Destination also serves as a source
One-address	OP src (implicit ACC)	LOAD a; ADD b; STORE x	Implicit accumulator register for all operations
Zero-address	OP (implicit stack)	PUSH a; PUSH b; ADD; POP x	Operands on a stack common model for stack-based bytecode such as the JVM.

Trade-off: Fewer operands → simpler, shorter instructions → but more instructions needed for the same computation.

1-, 2-, 3-addresses instructions

Example-3: What is the computation corresponding to the following 1-address instructions?

LOAD D
MUL E
STORE Y
LOAD A
MUL B
DIV Y
STORE Y

1 Address
LOAD M ($\text{Acc} \leftarrow M$)
STORE M ($M \leftarrow \text{Acc}$)
ADD M ($\text{Acc} \leftarrow \text{Acc} + M$)
SUB M ($\text{Acc} \leftarrow \text{Acc} - M$)
MUL M ($\text{Acc} \leftarrow \text{Acc} * M$)
DIV M ($\text{Acc} \leftarrow \text{Acc} / M$)

M is a memory address
Acc: accumulator

1-, 2-, 3-addresses instructions

Example-3: What is the computation corresponding to the following 1-address instructions?

LOAD	D	←	Load D to the accumulator
MUL	E	←	Multiply the value in the accumulator with E
STORE	Y	←	Store the result back to Y
LOAD	A	←	Load A to the accumulator
MUL	B	←	Multiply the value in the accumulator with B
DIV	Y	←	Divide the value in the accumulator by Y
STORE	Y	←	Store the result back to Y

$$Y = (A*B)/(D*E)$$

1-, 2-, 3-addresses instructions

Example-4: write the 3-address instructions which do the same as the following 1-address instructions? You can use the instructions available on the right.

LOAD D
MUL E
STORE Y
LOAD A
MUL B
DIV Y
STORE Y



Requirement: use general registers R0/R1/R2 to store intermediate result, so that value in A/B/C/D/E/F will not be overwritten

3 Address		
ADD	X, Y, Z	$(X \leftarrow Y + Z)$
SUB	X, Y, Z	$(X \leftarrow Y - Z)$
MUL	X, Y, Z	$(X \leftarrow Y * Z)$
DIV	X, Y, Z	$(X \leftarrow Y / Z)$

X, Y, Z: either memory address or registers

1-, 2-, 3-addresses instructions

Example-4: write the 3-address instruction which does the same as the following 1-address instructions?

$Y = (A*B)/(D*E)$

<pre>LOAD D MUL E STORE Y LOAD A MUL B DIV Y STORE Y</pre>		<pre>MUL R0 A B MUL R1 D E DIV Y R0 R1</pre>
--	--	--

Trade-off: Fewer operands → simpler, shorter instructions → but more instructions needed for the same computation.

Program counter and Instruction register

Example 5: The main memory contains a small program to be executed. Each instruction is stored at a separate memory address. Assume the instructions will be executed sequentially.

Q1: If the program counter contains an address value of 201, which instruction will be loaded in the IR to be executed next?

Program counter
201

Instruction register
???

```
200  MOV  R0,  b
201  SUB  R0,  c
202  ADD  R0,  #12
203  MOV  a,   R0
```

↑
memory
address

Program Counter (PC)

PC

Holds the address of the next instruction to fetch. Normally incremented after each fetch; modified by branch/jump instructions.

Instruction Register (IR)

IR

Holds the instruction currently being decoded and executed. The CU reads the IR to determine what operation to perform.

Program counter and Instruction register

Example 5: The main memory contains a small program to be executed. Each instruction is stored at a separate memory address. Assume the instructions will be executed sequentially.

Q1: If the program counter contains an address value of 201, which instruction will be loaded in the IR to be executed next?

Program counter
201

Instruction register
SUB R0, c

```
200 MOV R0, b
201 SUB R0, c
202 ADD R0, #12
203 MOV a, R0
```

↑
memory
address

Program Counter (PC)

PC

Holds the address of the next instruction to fetch. Normally incremented after each fetch; modified by branch/jump instructions.

Instruction Register (IR)

IR

Holds the instruction currently being decoded and executed. The CU reads the IR to determine what operation to perform.

Program counter and Instruction register

Example 5: The main memory contains a small program to be executed. Each instruction is stored at a separate memory address. Assume the instructions will be executed sequentially.

Q3: After executing the instruction currently contained in the IR register, which memory address should the PC contain?

Program
counter

???

Instruction
register

ADD R0, #12

```
200  MOV  R0,  b
201  SUB   R0,  c
202  ADD   R0, #12
203  MOV   a,  R0
```

↑
memory
address

Program Counter (PC)

PC

Holds the address of the next instruction to fetch. Normally incremented after each fetch; modified by branch/jump instructions.

Instruction Register (IR)

IR

Holds the instruction currently being decoded and executed. The CU reads the IR to determine what operation to perform.

Program counter and Instruction register

Example 5: The main memory contains a small program to be executed. Each instruction is stored at a separate memory address. Assume the instructions will be executed sequentially.

Q3: After executing the instruction currently contained in the IR register, which memory address should the PC contain?

Program counter
203

Instruction register
ADD R0, #12

```
200 MOV R0, b
201 SUB R0, c
202 ADD R0, #12
203 MOV a, R0
```

↑
memory
address

Program Counter (PC)

PC

Holds the address of the next instruction to fetch. Normally incremented after each fetch; modified by branch/jump instructions.

Instruction Register (IR)

IR

Holds the instruction currently being decoded and executed. The CU reads the IR to determine what operation to perform.

Sequential execution and branches

Example 5: What does the following 2-address instructions implement? Explain your result.

```
300  MOV  R0, a
301  MUL  R0, b
302  SUB  R0, c
303  BEQ  309
304  MOV  e, R0
    ⋮
309  do something
```

```
if (a*b-c)==0:
    do something
e = a*b-c
```

(A)

```
if (a*b-c)==0:
    do something
else:
    e = a*b-c
```

(B)

Sequential execution and branches

Example 5: What does the following 2-address instructions implement? Explain your result.

```
300  MOV R0, a  ← move the value of variable a to register R0: R0=a
301  MUL R0, b  ← Multiply the value of R0 with the value of variable b: R0 = a*b
302  SUB R0, c  ← Subtract the value of R0 with the value of variable c: R0 = a*b-c
303  BEQ 309    ← Branch to address 309, if the previous result is 0
304  MOV e, R0  ← Move the value of R0 to variable e
    :
309  do something
```

```
if (a*b-c)==0:
    do something
else:
    e = a*b-c
```

Flow of Control and Program Counter

Example 6: What should be the value in the PC after executing the instruction “BEQ 309” in the following two cases? Explain your result.

1) a=5, b=3, c=15.

2) a=5, b=3, c=20.

```
300  MOV R0, a
301  MUL R0, b
302  SUB R0, c
303  BEQ 309
304  MOV e, R0
    ⋮
309  do something
```

Flow of Control and Program Counter

Example 6: What should be the value in the PC after executing the instruction “BEQ 309” in the following two cases? Explain your result.

1) a=5, b=3, c=15.

2) a=5, b=3, c=20.

```
300  MOV R0, a
301  MUL R0, b
302  SUB R0, c
303  BEQ 309
304  MOV e, R0
    ⋮
309  do something
```

Answer:

- 1) a=5, b=3, c=15: $a*b=15$, $a*b-c=0$ is true, hence it will jump to address 309 and execute the instruction there, this is achieved by **updating the PC to 309**, since it points to the next instruction to be executed.
- 2) a=5, b=3, c=20: $a*b=15$, $a*b - c \neq 0$, hence the condition is false. In this case, there is no need to jump, it will simply follow the default sequential execution. Hence, **PC will remain 304** to reflect this.

Status Flags

After an ALU operation, the processor sets status flags (=bits) to indicate conditions such as zero, negative, carry, or overflow.

Zero

Z

Set to 1 if the result is exactly zero.

SUB R1, R1, R2 when $R1 = R2 \rightarrow Z = 1$

Negative / Sign

N

Set to 1 if the result is negative (MSB = 1 in two's complement).

SUB R1, R1, R2 when $R1 < R2 \rightarrow N = 1$

Carry

C

Set when unsigned arithmetic produces a carry out of the most significant bit (unsigned overflow).

Unsigned 8-bit: $200 + 100 > 255 \rightarrow C = 1$

Overflow

V

Set to 1 when a signed arithmetic result is outside the representable range.

Example: adding two large positive numbers produces a negative result.

How conditional branches work:

SUB R1, R1, R2 ; sets flags

BEQ target ; if Z flag = 1, jump to target address

Status flags

Example 7: What bit will be set in the Status register after executing the instruction “SUB R0, c” in the following two cases? Explain your result.

- 1) a=5, b=3, c=15.
- 2) a=5, b=3, c=20.

```
300  MOV R0, a
301  MUL R0, b
302  SUB R0, c
303  BEQ 309
304  MOV e, R0
```

N - set when the result was negative

Z - set when the result was zero

V - set when the result caused an overflow (for two's complement overflow)

C - set when the result caused a carry out of the leftmost bit (for unsigned overflow)

Status flags

Example 7: What bit will be set in the Status register after executing the instruction “SUB R0, c” in the following two cases? Explain your result.

- 1) a=5, b=3, c=15.
- 2) a=5, b=3, c=20.

```
300  MOV R0, a
301  MUL R0, b
302  SUB R0, c
303  BEQ 309
304  MOV e, R0
```

1) a=5, b=3, c=15: SUB R0, c performs $a*b-c=5*3-15=0$, so the result is 0. The Z bit will be set.

N - set when the result was negative

Z - set when the result was zero

V - set when the result caused an overflow (for two's complement overflow)

C - set when the result caused a carry out of the leftmost bit (for unsigned overflow)

Status flags

Example 7: What bit will be set in the Status register after executing the instruction “SUB R0, c” in the following two cases? Explain your result.

- 1) a=5, b=3, c=15.
- 2) a=5, b=3, c=20.

```
300  MOV R0, a
301  MUL R0, b
302  SUB R0, c
303  BEQ 309
304  MOV e, R0
```

1) a=5, b=3, c=20: SUB R0, c performs $a*b-c=5*3-20<0$, so the result is negative. The N bit will be set.

N - set when the result was negative

Z - set when the result was zero

V - set when the result caused an overflow (for two's complement overflow)

C - set when the result caused a carry out of the leftmost bit (for unsigned overflow)

Instruction execution

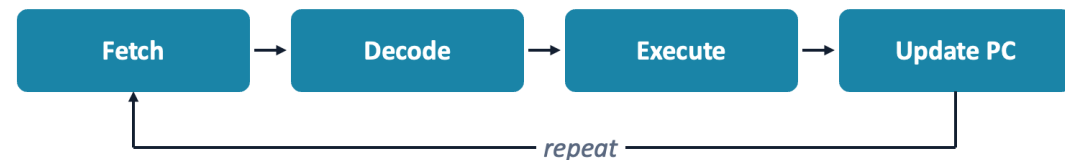
Example 8: Simulate the execution of the instructions assuming

1) a=5, b=3, c=15.

```
300 MOV R0, a
301 MUL R0, b
302 CMP R0, c
303 BEQ 309
304 MOV e, R0
```

The Instruction Cycle

The four-stage loop the CPU repeats for every instruction. Simplified view: register reads, memory access for load/store instructions, and result write-back are not shown as separate stages here.

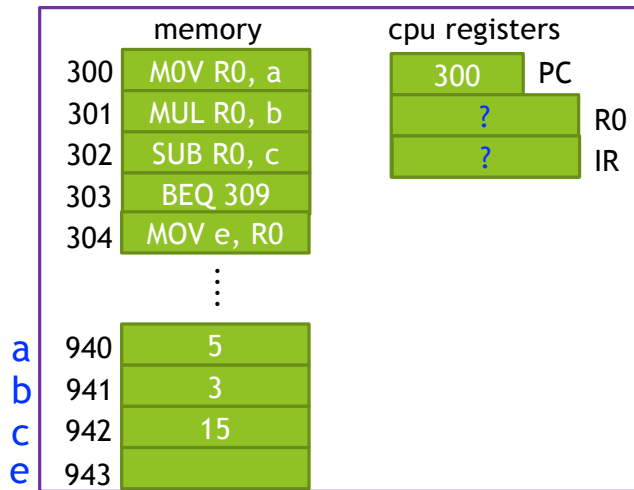


Fetch: Read the instruction at the **address in the PC** from memory into the IR.

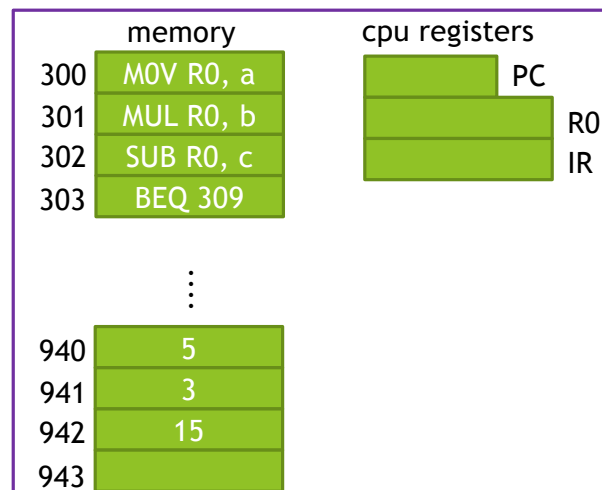
Decode: The Control Unit interprets the opcode and determines where the operands are located

Execute: The ALU or another unit carries out the operation (arithmetic, data move, branch).

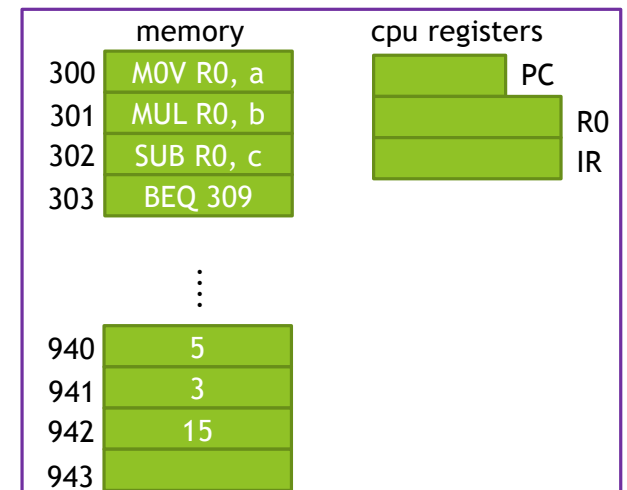
Update PC: the PC normally advances to the next instruction, by adding the size of the current instruction. A branch, call or return will instead replace it with a new address.



Step 1: fetch instruction



Step 2&3: decode + execute instruction

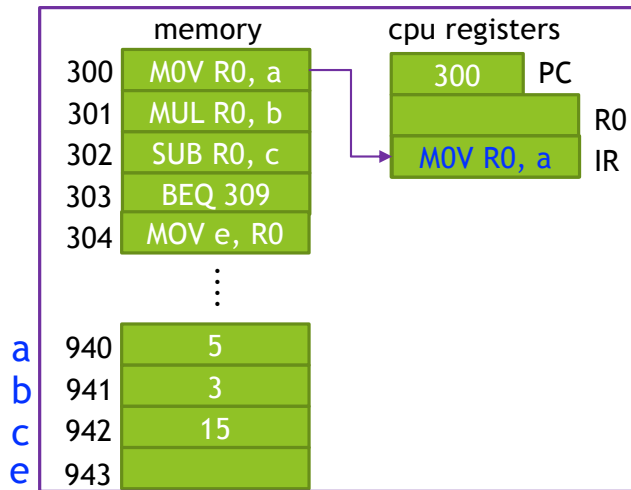


Step 4: update the PC

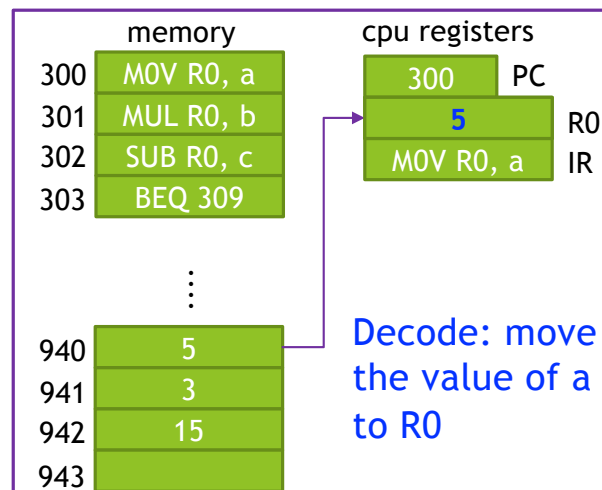
Update PC:

- Normally: $PC \leftarrow PC + \text{size of instruction}$, e.g. 1 here
- Branch: $PC \leftarrow \text{target address}$ (decided during Execute)

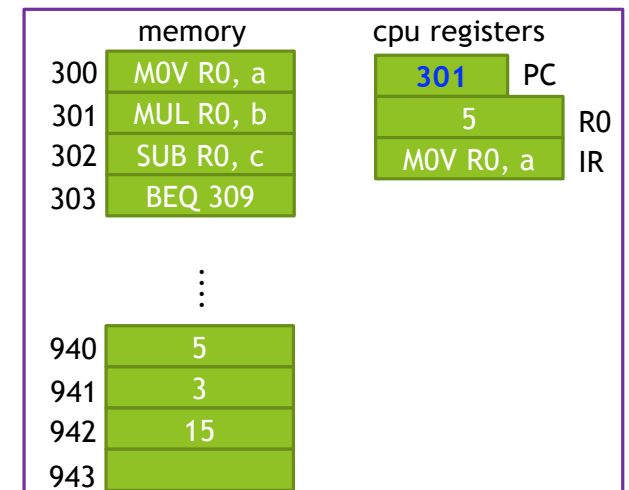
1) Execute the instruction pointed by the PC (300) - MOV R0, a



Step 1: fetch instruction

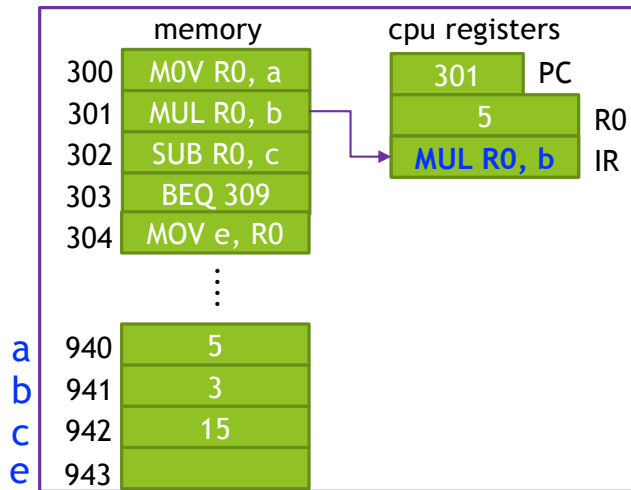


Step 2&3: decode + execute instruction

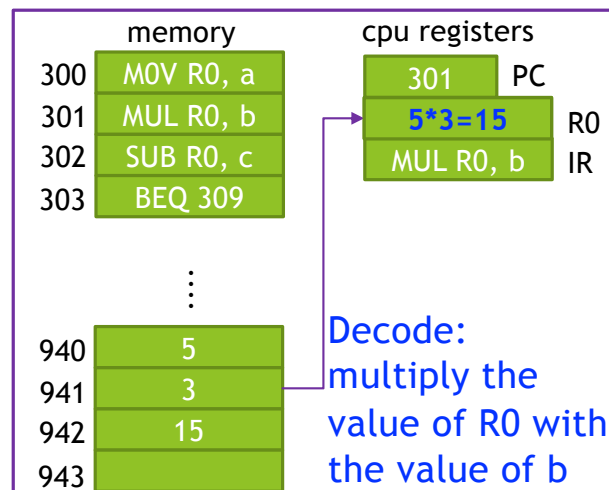


Step 4: update the PC

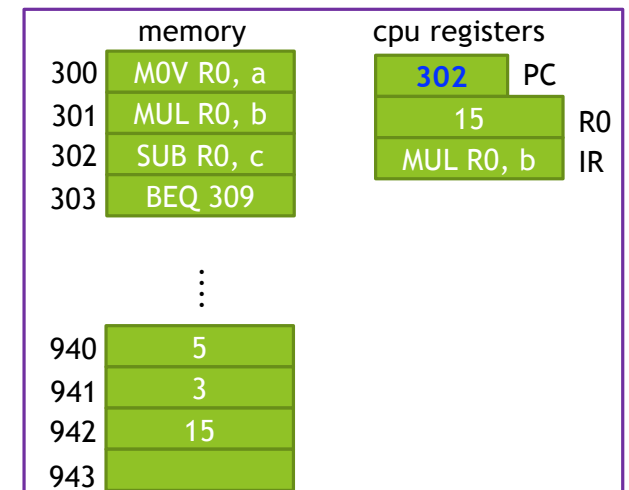
2) Execute the instruction pointed by the PC (301) - MUL R0, b



Step 1: fetch instruction

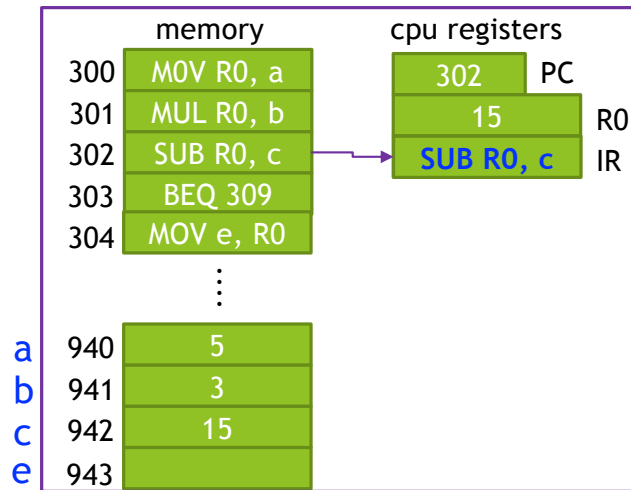


Step 2&3: decode + execute instruction

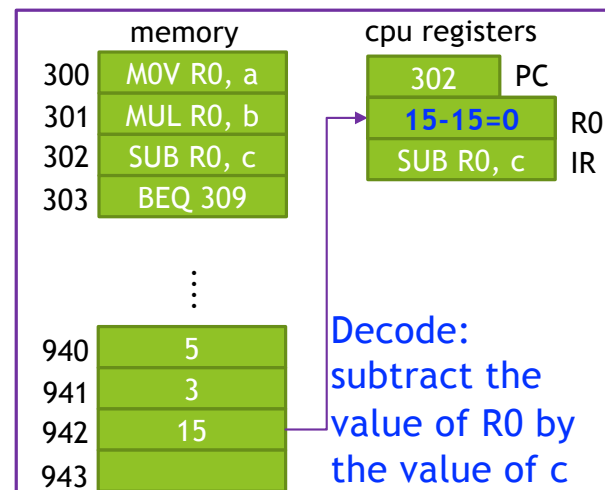


Step 4: update the PC

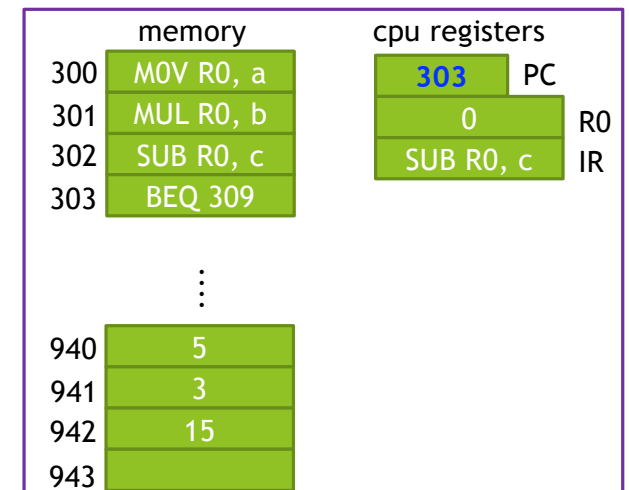
3) Execute the instruction pointed by the PC (302) - SUB R0, c



Step 1: fetch instruction

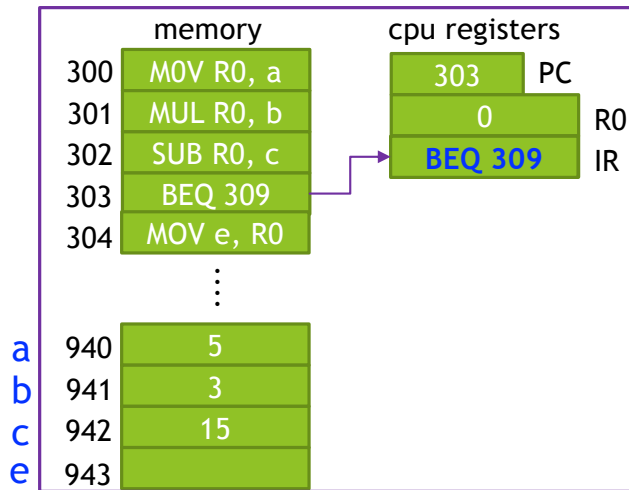


Step 2&3: decode + execute instruction

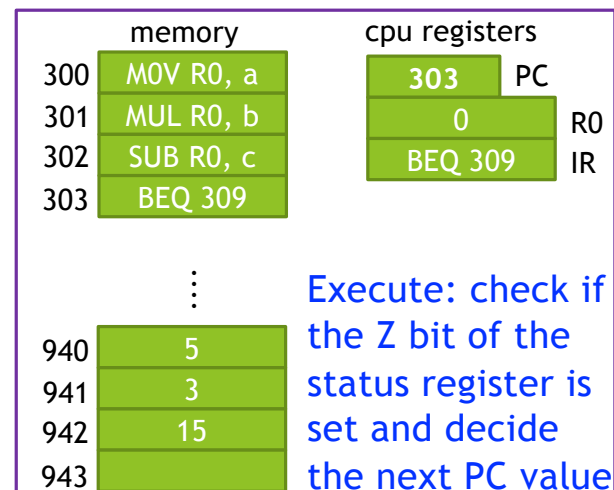


Step 4: update the PC

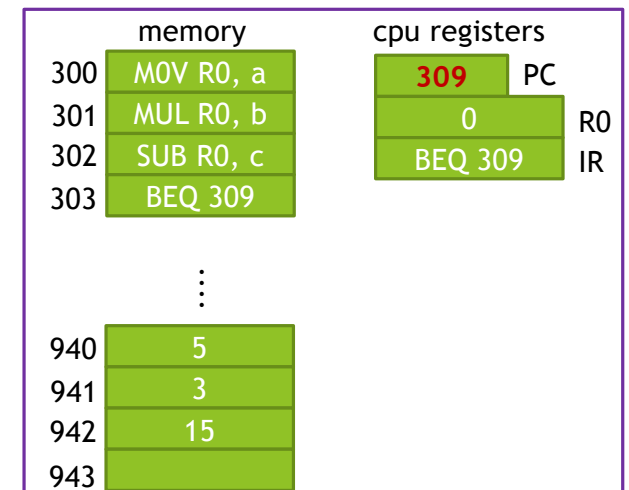
4) Execute the instruction pointed by the PC (303) - BEQ 309



Step 1: fetch instruction



Step 2&3: decode + execute instruction



Step 4: update the PC

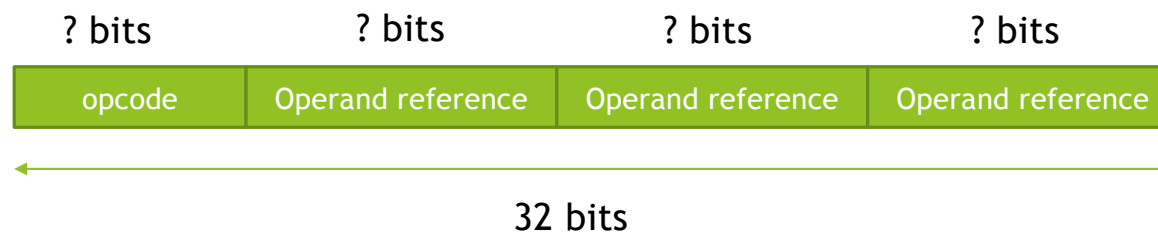
Update PC:

- Sequential: $PC \leftarrow PC + \text{size of instruction}$, e.g. 1 here
- Branch: $PC \leftarrow \text{target address (decided during Execute)}$

Exercise

Instruction format

Ex. 1: Assume a machine supports 3-address instructions. All instructions are 32 bits long, all operand references are of the same length as well, as shown below:



Q1: if the machine supports 256 different instructions, how many bits need to be reserved for the opcode field? Explain why?

Q2: What's the length for each operand reference? Explain why?

Q3: If operands are in main memory, how many different operand addresses can the machine refer to? Explain why?

Instruction type

Ex. 2: Write programs to compute $X = (A + B * C) / (D - E * F)$ for 3-address machine. The following instructions are available for use:

3 Address		
ADD	X, Y, Z	$(X \leftarrow Y + Z)$
SUB	X, Y, Z	$(X \leftarrow Y - Z)$
MUL	X, Y, Z	$(X \leftarrow Y * Z)$
DIV	X, Y, Z	$(X \leftarrow Y / Z)$

Requirement: use general registers R0/R1/R2 to store intermediate result, so that value in A/B/C/D/E/F will not be overwritten

X, Y, Z: either memory address or registers

Q: Include your program, and count how many instructions are needed?

Instruction type

Ex. 3: Write programs to compute $X = (A + B * C) / (D - E * F)$ for 2-address machine. The following instructions are available for use:

2 Address
MOVE X, Y ($X \leftarrow Y$)
ADD X, Y ($X \leftarrow X + Y$)
SUB X, Y ($X \leftarrow X - Y$)
MUL X, Y ($X \leftarrow X * Y$)
DIV X, Y ($X \leftarrow X / Y$)

Requirement: use general registers R0/R1/R2 to store intermediate result, so that value in A/B/C/D/E/F will not be overwritten

X, Y: either memory address or registers

Q: Include your program, and count how many instructions are needed?

Instruction type

Ex. 4: Write programs to compute $X = (A + B * C) / (D - E * F)$ for 1-address machine. The following instructions are available for use:

1 Address
LOAD M (Acc $\leftarrow$ M)
STORE M (M $\leftarrow$ Acc)
ADD M (Acc $\leftarrow$ Acc + M)
SUB M (Acc $\leftarrow$ Acc - M)
MUL M (Acc $\leftarrow$ Acc * M)
DIV M (Acc $\leftarrow$ Acc / M)

Hint: you can use variable X to store intermediate result if needed

M is a memory address
Acc: accumulator

Q: Include your program, and count how many instructions are needed?

Sequential execution and branches

Ex. 5: What is the computation corresponding to the following assemble code? Derive a small piece of python code corresponding to it. Express it with A, B and C.

200	MOV R1, A	← move the value of variable A to register R1
201	MOV R2, B	← move the value of variable B to register R2
202	CMP R1, R2	← compare R1 against R2
203	BRLT 207	← if R1 is less than R2, jump to address 207
204	SUB R1, R2	← $R1 = R1 - R2$
205	MOV R3, R1	← Move the value in R1 to R3
206	BR 209	← Unconditionally branch to 209
207	SUB R2, R1	← $R2 = R2 - R1$
208	MOV R3, R2	← Move the value in R2 to R3
209	MOV C, R3	← Write the value in R3 back to variable C

↑
address

R1, R2, R3: general registers

A, B, C: memory corresponding to variable A, B, C

CMP R1 R2: compare instruction, compare R1 against R2

BRLT: branch if less than, based on the result of previous instruction

Sequential execution and branches

Ex. 6: Which instructions will be executed if A = 15, B = 8? What's the value in C? Explain your results.

200	MOV R1, A	← move the value of variable A to register R1
201	MOV R2, B	← move the value of variable B to register R2
202	CMP R1, R2	← compare R1 against R2
203	BRLT 207	← if R1 is less than R2, jump to address 207
204	SUB R1, R2	← R1=R1-R2
205	MOV R3, R1	← Move the value in R1 to R3
206	BR 209	← Unconditionally branch to 209
207	SUB R2, R1	← R2=R2-R1
208	MOV R3, R2	← Move the value in R2 to R3
209	MOV C, R3	← Write the value in R3 back to variable C

↑
address

R1, R2, R3: general registers

A, B, C: memory corresponding to variable A, B, C

CMP R1 R2: compare instruction, compare R1 against R2

BRLT: branch if less than, based on the result of previous instruction

Program counter

Ex. 7: What's the value in the program counter after executing the first 4 instructions and in the case that A = 11, B = 20 ? Explain your result.

200	MOV R1, A	PC = ?
201	MOV R2, B	PC = ?
202	CMP R1, R2	PC = ?
203	BRLT 207	PC = ?
204	SUB R1, R2	
205	MOV R3, R1	
206	BR 209	
207	SUB R2, R1	
208	MOV R3, R2	
209	MOV C, R3	
↑		
address		

Status flag

Ex. 8: which bit(s) of the status register will be set after executing the instruction corresponding to $129_{10} + 135_{10}$ for 8-bit unsigned addition. Explain why?

N - set when the result was negative

Z - set when the result was zero

V - set when the result caused an overflow (for two's complement overflow)

C - set when the result caused a carry out of the leftmost bit (for unsigned overflow)